

金融领域智能问答机器人系统



技术架构与实现解析

INTELLIGENT Q&A ROBOT SYSTEM IN FINANCIAL SECTOR

目录CONTENTS

01

系统概述

核心功能与价值
System Overview

02

技术架构

系统分层与模块关系
Technical Architecture

03

核心模块详解

从数据模型到API服务
Core Modules

04

关键技术解析

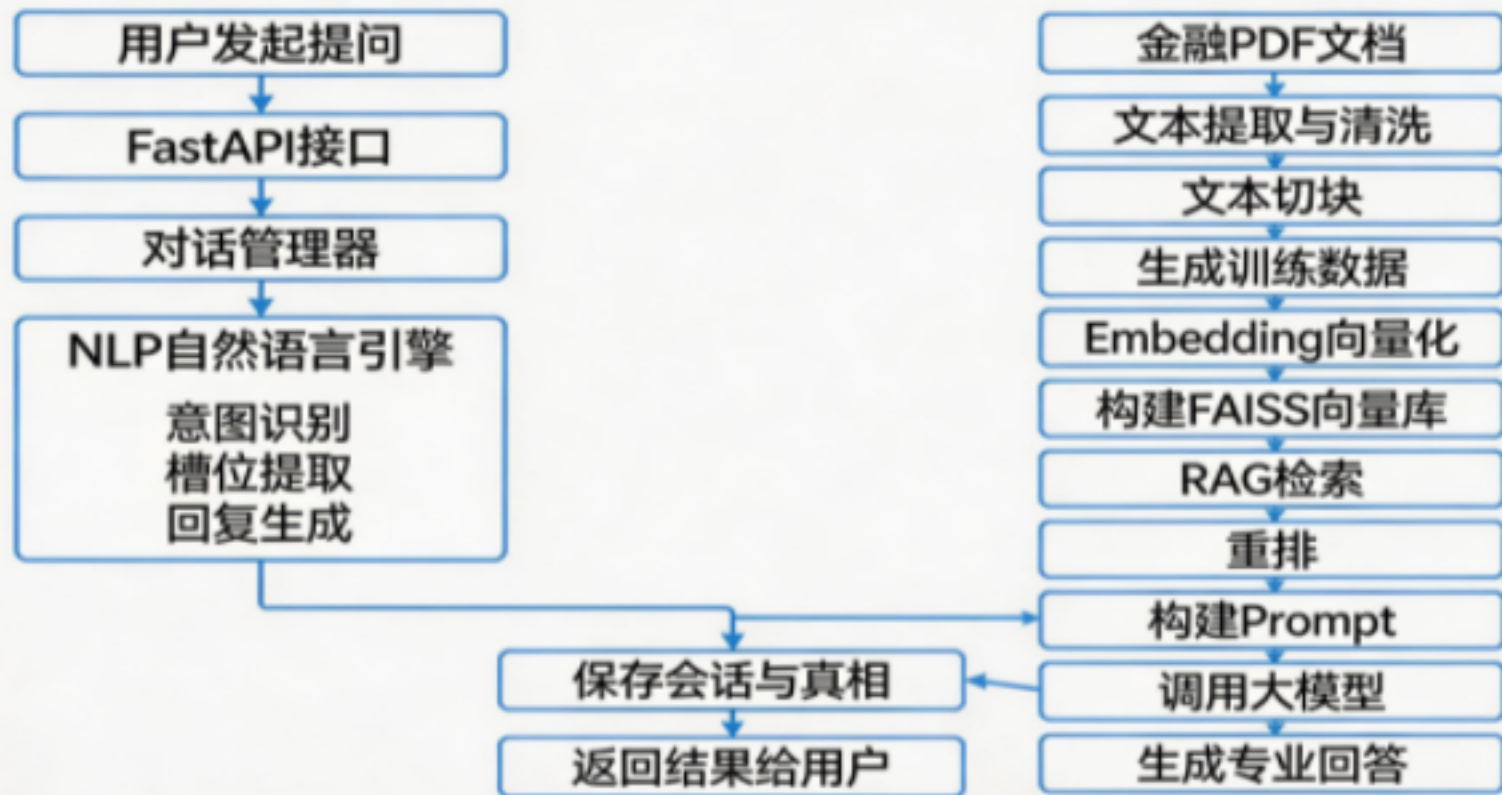
RAG 检索增强生成与大模型微调技术
Key Technology Analysis

05

工作流程与总结

端到端实例演示与未来发展展望
Workflow & Summary

系统整体流程图



01 系统概述：核心功能

本系统是一个专为金融领域设计的智能问答机器人，旨在提供精准、专业的自动化咨询服务，赋能金融机构实现服务效率与用户体验的双重升级。



意图识别

精准识别用户在股票、基金、保险、投顾等全金融场景下的多样化咨询意图。



信息提取

基于NLP技术自动抽取问题中的关键要素，如标的代码、时间区间、投资金额等。



专业回复

深度融合金融预制知识库与大模型生成能力，输出严谨、准确、合规的回复。



会话管理

智能维护多轮对话上下文，确保交互逻辑连贯，提供自然流畅的拟人化体验。



数据持久化

完整记录全量会话、消息内容与交互行为数据，支持后续业务分析与模型优化。



API 服务

提供标准、稳定的RESTful HTTP接口，便于与各类Web端、App等前端应用快速集成。

01 系统概述：核心价值



轻量化与低成本

基于正则表达式实现意图识别，无需复杂的NLP模型训练，部署成本低，易于维护。



领域专业性

内置覆盖股票、基金、保险、投顾等多个金融场景的专业知识库和预制回复，确保答案的准确性和权威性。



高扩展性

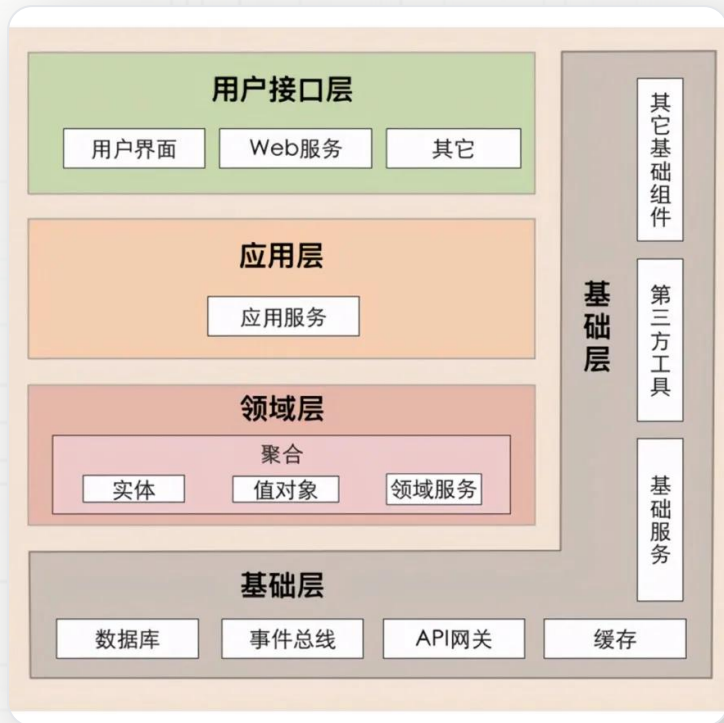
支持RAG（检索增强生成）和大模型微调，可轻松接入自定义金融知识库，持续提升系统的专业能力。



工程化与可用性

提供完整的会话管理、数据持久化和API服务，可直接作为后端服务对接前端应用，快速上线。

02 技术架构：系统分层



表现层 (API层) | 文件: `app.py`

基于FastAPI框架构建，对外提供HTTP API接口，统一处理用户请求与返回响应结果。

业务逻辑层 | 文件: `dialogue_manager.py`, `nlp_engine.py`

系统核心中枢，统筹对话管理、意图精准识别、关键槽位自动填充及智能回复生成逻辑。

数据访问层 | 文件: `database.py`

封装底层数据交互逻辑，负责与SQLite数据库对接，实现会话与配置数据的持久化存储与查询。

基础模型层

文件: `models.py`
定义系统中所有核心数据结构，包括意图定义、消息格式及会话实体模型。

工具层

文件: `prompt.py`, `*.py`
提供底层支撑能力，涵盖RAG检索增强、大模型API调用、PDF解析及向量库构建。

02 技术架构：模块交互流程

01



用户请求

用户通过 `/chat` 接口发送问题，作为整个交互流程的起点。

02



API 接入层 (app.py)

作为系统的统一入口，负责接收 HTTP 请求并将其转发给对话管理器进行处理。

03



对话管理器

核心的会话中枢，负责维护上下文，并根据逻辑调用对应的 NLP 引擎模块处理输入。

04



NLP 引擎 (nlp_engine.py)

执行意图识别与槽位提取：高置信度直接回复；低置信度时则触发 RAG 增强检索。

05



RAG 增强生成 (prompt.py)

针对低置信度问题，从自有知识库中检索相关片段，并结合大模型生成准确的增强回复。

06



数据持久化 (database.py)

将会话 ID、提问内容、回复内容及时间戳完整保存至数据库，支持后续追溯与分析。

07



返回最终响应

API 层将生成好的最终回复，封装成标准的 JSON 格式，返回给前端客户端展示给用户。

03 核心模块详解：`models.py` - 数据模型定义

`models.py` 是整个系统的数据基石，定义了业务逻辑流转所需的所有核心数据结构与类型约束。



枚举类 (Enums) · 定义固定类型

- **Intent:** 定义用户意图类型，例如：`STOCK\_QUERY` (股票查询)、`FUND\_QUERY` (基金查询)。
- **DialogueState:** 定义对话生命周期状态，如 `START`、`INTENT\_CONFIRMED`、`SLOT\_FILLING`。
- **MessageRole:** 区分消息来源，固定为 `USER` (用户) 和 `BOT` (机器人)。



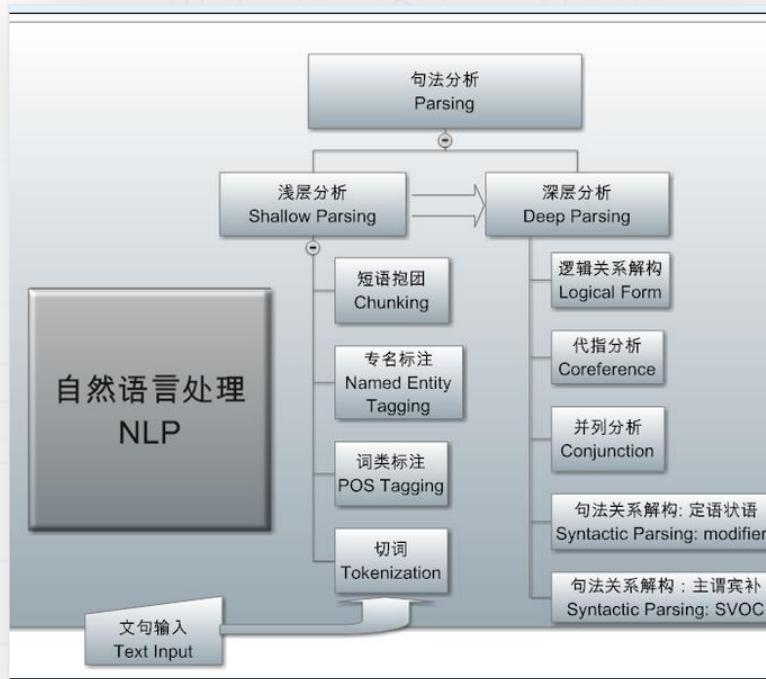
Pydantic Models · 结构化数据载体

- **基础单元:** `Message` (单条消息实体)、`Slot` (关键信息槽位，如股票代码/基金名称)。
- **会话上下文:** `ConversationSession` 封装多轮对话状态，用于维护上下文记忆。
- **接口契约:** `ChatRequest` / `ChatResponse` 规范前后端与微服务间的交互数据格式。



系统核心价值：通过严格的类型定义与数据校验，确保数据在系统各模块间传递的一致性与有效性，是实现全链路“类型安全”与“防错容错”的底层逻辑基础。

03 核心模块详解：nlp_engine.py - NLP核心引擎



IntentRecognizer (意图识别)

- 基于正则匹配用户输入，精准识别意图（如“查股票”）。
- 计算匹配置信度，为后续流程提供可靠决策依据。



SlotFiller (槽位填充)

- 从用户的自然语言中提取关键信息（即“槽位”）。
- 例如：股票代码“600000”、时间范围“今天”等具体参数。



ResponseGen (回复生成)

- 根据识别的意图返回预设的专业金融领域回复。
- 支持动态将提取的槽位信息填充至回复模板，生成个性化答案。



NLPEngine (NLP引擎)

- 系统的“大脑”：整合上述三个组件，提供统一NLU接口。
- 智能决策：根据输入复杂度判断是否需调用RAG增强生成。

03 核心模块详解：`dialogue\_manager.py` - 对话管理器

`dialogue\_manager.py` 作为对话系统的“大脑中枢”，负责统筹多轮对话的全生命周期，精准维护上下文状态，保障人机交互的逻辑连贯与响应流畅。



会话管理 (Session Management)

- 负责会话的全周期管理：创建新会话、动态更新状态及自动清理过期会话。
- 持久化存储关键信息：包括当前对话意图、已成功匹配并填充的槽位等上下文元数据。



对话处理 (Dialogue Processing)

- 中枢调度：接收用户输入并转发至NLP引擎进行意图识别与实体提取。
- 决策与响应：基于NLP分析结果更新会话状态，并最终生成逻辑连贯的回复反馈给用户。



上下文维护 (Context Maintenance)

- 记忆保持：完整记录并保存用户与机器人的交互历史，确保多轮对话的上下文连贯性。
- 资源优化：动态控制历史消息的缓存长度，防止因无限增长导致的系统内存溢出风险。



降级处理 (Fallback Mechanism)

- 兜底策略：当NLP引擎无法识别意图或生成有效回复时，系统自动触发降级逻辑。
- 增强生成：无缝切换并调用RAG (检索增强生成) 接口进行辅助回答，最大化保障服务的可用性与回答质量。

03 核心模块详解：database.py - 数据库持久化



核心定位：数据基石

`database.py` 模块基于轻量级、文件式的SQLite引擎，为对话系统提供完整的数据持久化能力。它将内存中的瞬时会话、消息与状态映射为磁盘上的持久存储，既保证了服务重启后的数据不丢失，也为后续的业务分析与优化提供了可追溯的数据源。



数据库初始化 (Schema)

在启动时自动检查并创建三张核心数据表：

- **sessions**：用户会话元数据
- **messages**：交互消息全量流水
- **dialogue_states**：对话状态与槽位快照



核心数据操作 (CRUD)

封装基础的增删改查接口，支撑业务流转：

- 新用户会话的创建与标识
- 消息与状态的实时落盘
- 按会话ID检索完整上下文历史
- 支持多会话的并发读写



数据统计与业务洞察 (Analytics)

提供多维度统计接口，挖掘数据资产价值：

- **运营指标**：查询平台累计总会话数、消息总量及活跃用户数，量化系统规模。
- **业务优化**：基于历史消息分析高频问题、识别对话瓶颈，辅助对话策略优化。

03 核心模块详解：`app.py` - FastAPI服务接口



系统入口与基石

`app.py` 是整个应用的启动文件与对外服务的统一出口。它基于高性能的FastAPI 框架构建，负责接收前端请求、路由分发及结果返回，是连接用户与后端逻辑的关键桥梁。

/chat (核心问答)

处理用户的自然语言提问，整合多模型逻辑返回精准回复。

/health (健康检查)

监控服务状态，返回CPU、内存及依赖服务的连通性指标。

/rag-chat (纯RAG检索)

绕过复杂逻辑，直接调用检索增强生成模块，提供最相关的事实性回答。

/statistics (数据统计)

汇总系统运行数据，如总会话数、活跃用户数及平均响应时间。

/compare (模型效果对比)

输入同一问题，同时调用基座模型与微调模型，直观对比回复质量。

/session/{id} (会话管理)

支持对特定用户会话进行历史记录查询、重置或删除操作。



自动生成Swagger文档



CORS 跨域支持



异步IO 极致性能

03 核心模块详解：prompt.py - RAG检索增强生成

核心工作流程

1

加载向量库

加载预构建的FAISS库，内含海量金融领域专业知识数据。

2

语义检索

用户提问向量化，在库中快速检索语义最相似的文档片段。

3

结果重排

CrossEncoder模型重排，精准筛选Top-K个最相关片段。

4

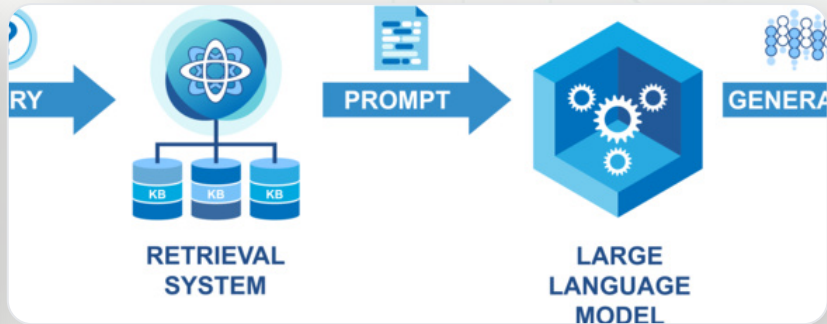
构建 Prompt

整合上下文信息与用户问题，生成结构化的专业提示词。

5

调用大模型

将 Prompt 发送给大模型 (Qwen2.5-7B)，生成基于知识库的增强回复。



模块核心价值

解决了基础大模型的局限性，赋予模型回答其训练数据之外、时效性强的金融市场信息或特定垂直领域专业问题的能力，确保生成内容的准确性与专业性。

04 关键技术解析：大模型微调

1.py - 微调数据生成

功能：处理金融PDF文档，提取有效信息并进行清洗与结构化处理。

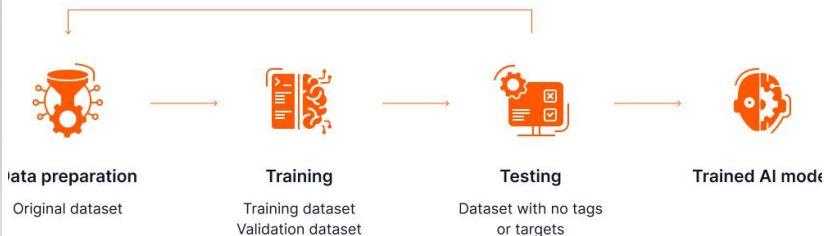
流程：1. 提取PDF文本 → 2. 清洗乱码、页码等噪声 → 3. 切分200-400字短片段 → 4. 生成 `instruction-input-output` 格式JSON数据集。

2.py - 向量库构建

功能：将非结构化的金融PDF文档转换为结构化的向量库，支撑RAG实时检索。

流程：1. 清洗并加载多文档 → 2. 500字/块切分并设置重叠 → 3. 使用 `BAAI/bge-large-zh-v1.5` 模型向量化 → 4. 构建并持久化FAISS向量库。

Model training workflow



技术价值

通过这两个核心脚本，实现了“微调数据准备”与“RAG检索底座”的自动化构建，不断用最新的金融知识增强模型的专业性与准确性。

04 关键技术解析：大模型微调参数

为了让模型更好地适应金融领域的专业问答需求，我们基于开源模型进行了针对性的微调优化，以下是核心的技术参数设置方案。

基础架构配置

模型名称	模型路径	模型名称
Qwen2.5-7B-Instruct	Qwen/Qwen2.5-7B-Instruct	huggingface
训练方法	训练点路径	
量化方法 (Optional)	量化方法	量化方法
bmb	qwen	none
训练精度		

模型基座：Qwen2.5-7B-Instruct
微调方法：LoRA (Low-Rank Adaptation)
量化策略：4-bit (QLoRA) 高效量化
计算精度：bfloat16 (平衡性能与显存)

核心训练参数

学习率	梯度范数	最大梯度范数	最大采样
1.0	1.0	1000	
训练批次	训练步数	训练步数	训练步数
1	1	1000	1000
训练步数	训练步数	训练步数	训练步数
1	1	1000	1000
训练步数	训练步数	训练步数	训练步数
1	1	1000	1000

学习率 / Epochs: 5e-5 / 1.0
梯度范数 / 采样数: 1.0 / 10,000
文本截断长度: 2048 tokens
Batch Size / 梯度累积: 2 / 4

总结：上述参数在保证模型金融专业知识准确性的同时，最大化了训练的计算效率与稳定性。

04 关键技术解析：微调效果对比

通过微调，模型在金融领域的专业知识深度与回答的实用性上均实现了质的飞跃。

● 基座模型 (Qwen2.5-7B)

通货膨胀是指货币供应量超过经济实际需求，导致货币贬值、物价普遍上涨的现象。在通货膨胀期间，货币的购买力下降，即同样的货币可以购买的商品和服务比变少。通货膨胀会对经济产生多方面的影响，包括影响消费者和投资者的购买力、影响企业的成本和价格决策、影响政府的财政和货币政策等。在高通货膨胀率的情况下，股票的表现通常并不理想，因为股票的价值与公司的盈利能力、市场预期等因素密切相关，而这些因

● 微调模型 (Qwen2.5-7B + LoRA)

通货膨胀是指货币供应量超过经济实际需求，导致一般价格水平持续上涨的现象。这种价格上涨会降低货币的购买力，即同样的货币在通货膨胀期间能购买的商品和服务变少了。通货膨胀通常用消费者价格指数 (CPI) 或生产者价格指数 (PPI) 等指标来衡量。

在参考资料中提到的20世纪70年代的通货膨胀，以及德国在20世纪初经历的高通货膨胀率，都是通货膨胀

Q: 什么是通货膨胀?

◆ **基座模型**：给出教科书式的泛泛定义，缺乏实际场景的关联。

✅ **微调模型**：结合70年代滞胀等历史案例，分析了对投资者、普通工薪族及借贷者的差异化影响，回答具备实战参考价值。

正在对比两个模型的回答...

什么是经济

● 基座模型 (Qwen2.5-7B)

经济是指一个国家或地区内生产、分配、交换和消费商品和服务的体系。它包括了生产资源的使用、商品和服务的生产、分配、交换和消费等一系列活动。经济活动

● 微调模型 (Qwen2.5-7B + LoRA)

经济是指一个国家或地区内生产、分配、交换和消费商品和服务的体系。它包括了生产资源的使用、商品和服务的交换以及收入的分配。经济活动涉及个人、企业、

Q: 什么是经济?

◆ **基座模型**：仅描述了生产、分配、交换、消费等基础要素，内容浅显。

✅ **微调模型**：进一步阐述了经济运作的根本目的（满足社会需求、增进社会福利）及市场与计划两种主要机制，回答更加全面且逻辑严密。



核心结论：微调后的模型成功摆脱了“泛泛而谈”的局限，能够结合真实金融场景，提供兼具理论深度与实践指导意义的高质量专业回答。

05 系统工作流程示例



用户问题：“查询 600000 股票的今日行情”

01 请求进入

用户通过系统前端界面
发送问题

02 意图识别

NLP引擎识别出用户意
图为“股票查询”

03 信息提取

自动提取股票代
码“600000”等关键信
息

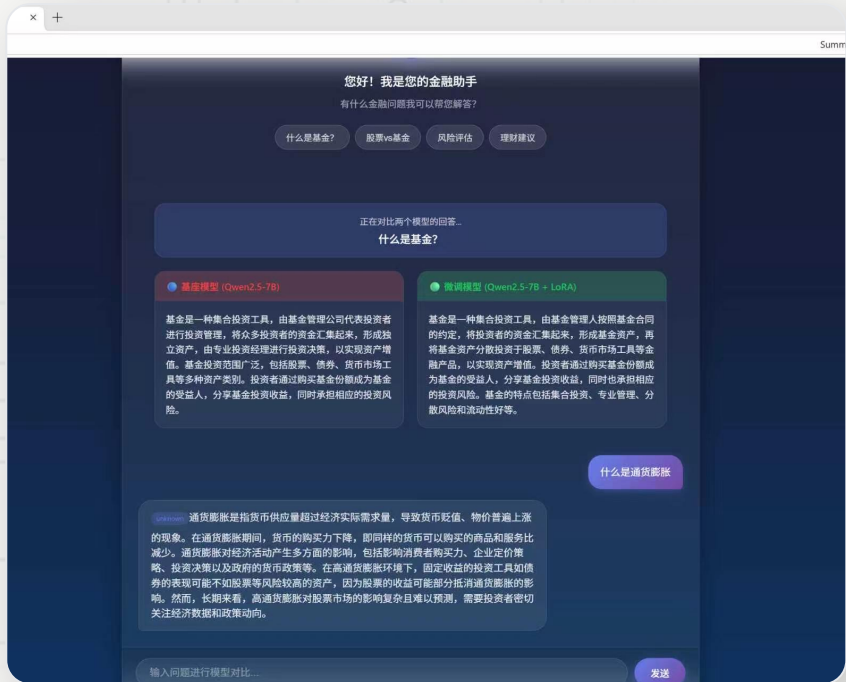
04 生成回复

结合预制知识库，生成
专业的投资建议

05 返回结果

将结构化结果展示在前
端界面上

05 系统工作流程示例：效果展示



模型对比功能

同时查看基座模型和微调模型的回答差异，直观对比优化效果，辅助决策参考。



完整对话历史记录

完整记录用户与机器人的每一轮交互过程，支持回溯查询，确保业务逻辑可追溯。



高频场景快捷提问

针对常见金融业务场景，内置快捷提问入口，降低用户操作成本，提升咨询效率。

05 总结与展望

总结 SUMMARY



技术成熟

采用分层架构，模块职责清晰，技术栈成熟稳定，确保系统可靠运行。



功能完备

覆盖从NLU意图识别到RAG知识检索的全流程，全面满足金融问答核心需求。



易于扩展

灵活支持RAG知识库扩展与模型微调，可快速适配新的业务场景和需求。



工程化部署

提供完整的API接口与数据持久化方案，具备高可用性，可直接用于生产环境。

展望 OUTLOOK



引入先进NLU模型

探索使用BERT等预训练模型替代正则，提升意图识别的准确率和泛化能力。



RAG流程优化

引入更高效的检索算法和重排模型，进一步优化知识库的利用效率与问答质量。



个性化推荐

基于用户画像与历史对话记录，提供千人千面的个性化金融投资建议。



多模态交互

拓展交互边界，支持语音输入、图表输出等多种交互方式，提升用户体验。



感谢聆听

THANKS FOR LISTENING

期待与您深入交流